# Analysis of Algorithms — Unit-wise Detailed Explanations

---

## Unit I — Performance Analysis & Mathematical Background

### 1. Growth of functions and Asymptotic Notations (Big-O, Big-Ω, Θ)

**When to use:**
We measure how running time or memory use grows as input size `n` becomes large. This ignores constants and lower-order terms and focuses on the dominant growth. Use these notations to compare algorithms independent of machine details.

**What they mean :**

- **Big-O (O):** an upper bound — the algorithm's running time will not grow faster than this rate for sufficiently large `n`. Use it to describe worst-case behavior.
- **Big-Ω (Ω):** a lower bound — the running time grows at least this fast for sufficiently large `n`. Use it to describe best-case.
- **Θ (Theta):** a tight bound — both an upper and lower bound; the function grows at this rate asymptotically.

**Worked example:**
For `T(n) = 3n^2 + 5n + 10`: as `n` grows, the `3n^2` term dominates. So `T(n) = Θ(n^2)`. Concretely, for large `n` there exist constants `c1, c2, n0` such that `c1 n^2 ≤ T(n) ≤ c2 n^2` for all `n ≥ n0`.

**Practical note:** Know how to simplify expressions (drop lower order terms and constants).

---

### 2. Master Method (for divide & conquer recurrences)

**When to use:**
When you have divide-and-conquer recurrences of the form `T(n) = a T(n/b) + f(n)`, the Master Method gives the asymptotic solution by comparing `f(n)` with `n^{log_b a}`.

**Approach :**

Syntax Syndicate

1. Compute `p = log_b a` (the work in subproblems combined if there were no extra `f(n)`).

2. Compare `f(n)` to `n^p`:

    - If `f(n)` grows **slower** than `n^p` (polynomially), the cost is dominated by subproblems → `T(n) = Θ(n^p)`.

    - If `f(n)` is **same order** (`Θ(n^p)`), you multiply by `log n` → `T(n) = Θ(n^p log n)`.

    - If `f(n)` grows **faster** (polynomially larger), and a regularity condition holds, `f(n)` dominates → `T(n) = Θ(f(n))`.

**Worked examples:**

- `T(n) = 2 T(n/2) + n` → `a=2, b=2, p = log_2 2 = 1`, `f(n)=n = Θ(n^p)` → middle case → `T(n)=Θ(n log n)`. (Merge sort.)

- `T(n) = 3 T(n/4) + n` → `p = log_4 3 ≈ 0.79`, `f(n)=n` grows faster than `n^p` → `T(n)=Θ(n)` (dominant `f(n)`).

**Practical note:** Master Method applies only when recurrence fits the standard form and `f(n)` is "nice" (polynomial factors). If not, use recursion tree or substitution.

---

# Unit I — Sorting algorithms

## Selection Sort — detailed approach & example

**How it works :**
Repeat: find the minimum element in the unsorted suffix and swap it into the next position in the sorted prefix. After `i` iterations, first `i` elements are the `i` smallest in sorted order.

**Step-by-step example:** `[64, 25, 12, 22, 11]`

1. Entire array: minimum is `11`. Swap with first element: `[11, 25, 12, 22, 64]`.

2. Consider suffix starting at position 2: `[25,12,22,64]`. Minimum `12`: swap into position 2 → `[11,12,25,22,64]`.

Syntax Syndicate

3. Continue until sorted: final `[11,12,22,25,64]`.

### Selection Sort — Algorithm

1. Start with index `i = 0` (first position).
2. Scan the array from `i` to end to find the index of the minimum element.
3. Swap that minimum element into position `i`.
4. Increment `i` (i = i + 1).
5. Repeat steps 2–4 until `i` reaches the second last element.
6. Array is sorted.

**Complexity & notes:**

- Comparisons: roughly `n(n-1)/2`. Time: **O(n²)** in all cases. Space: **O(1)**. Not adaptive and not stable by default. Use only for small arrays or when memory is very tight.

---

## Insertion Sort — detailed approach & example

**How it works :**
Treat the left part of the array as sorted. Take the next element (the "key"), shift larger elements in the sorted prefix to the right, and insert the key into its correct place.

## Insertion Sort — Algorithm

1. Treat first element as a sorted subarray.
2. For each next element (call it `key`), compare `key` with elements in the sorted subarray from right to left.
3. Shift all larger elements one position to the right to create a slot.
4. Insert `key` into the slot where elements to left ≤ `key` and to right > `key`.
5. Move to next element and repeat until whole array processed.

**Step-by-step example:** `[5, 3, 4, 1]`

Syntax Syndicate

1. Start with `5` (sorted prefix `[5]`). Insert `3`: shift `5` right and put `3` at front → `[3,5,4,1]`.

2. Insert `4`: shift `5` → `[3,4,5,1]`.

3. Insert `1`: shift `3,4,5` → `[1,3,4,5]`.

**Complexity & notes:**

- Best case (already sorted): **Ω(n)** (only one pass).
- Worst case (reverse sorted): **O(n²)**.
- Space: **O(1)**. Stable and adaptive — very efficient for small arrays or nearly sorted data (used in practice as final pass in hybrid sorts).

---

# Unit II — Divide & Conquer

## Divide & Conquer paradigm

**When to use:**
Divide & Conquer splits a problem into smaller subproblems, solves subproblems recursively, then combines their solutions. Use when the problem can be broken into independent subproblems of the same form.

**Typical steps:**

1. Divide input into two or more smaller parts.
2. Recursively solve each part.
3. Combine results (merge, partition recomposition, etc.).

**Complexity idea:** The recurrence `T(n) = a T(n/b) + f(n)` expresses the work: `a` subproblems of size `n/b` plus combination cost `f(n)`.

---

## Merge Sort

### Merge Sort — Algorithm

Syntax Syndicate

1. If array length ≤ 1, it is already sorted — stop.
2. Split the array exactly into two halves.
3. Recursively apply merge sort to the left half.
4. Recursively apply merge sort to the right half.
5. Merge the two sorted halves:
    - Repeatedly pick the smaller front element from the two halves and append to result.
    - When one half is empty, append the remainder of the other half.
6. Result is the merged sorted array.

**How it works :**

1. If array has one element, it's sorted.

2. Otherwise, split array into two halves.

3. Recursively sort left half and right half.

4. Merge the two sorted halves by repeatedly choosing the smaller front element of the two halves until one is empty, then append the remainder.

**Worked example:** `[38, 27, 43, 3, 9, 82, 10]`

- Split into `[38,27,43,3]` and `[9,82,10]`

- Recursively split until singletons: `[38] [27] [43] [3]` and `[9] [82] [10]`

- Merge siblings to produce sorted subarrays: `[27,38] [3,43]` → then merge to `[3,27,38,43]`; other side → `[9,10,82]`

- Merge these two sorted halves → `[3,9,10,27,38,43,82]`.

**Complexity & notes:**

- Time: **Θ(n log n)** in all cases (balanced recurrence).

- Space: **O(n)** extra for merging (unless careful in linked lists).

- Very stable and predictable; used for external sorting and very large data sets.

---

Syntax Syndicate

# Quick Sort

## Quick Sort — Algorithm

1. If subarray has 0 or 1 elements, it is sorted — stop.
2. Choose a pivot element (last / first / median / random).
3. Partition the array so that:
    - All elements ≤ pivot are placed to pivot's left,
    - All elements ≥ pivot to pivot's right,
    - Pivot lands in its final sorted position.
4. Recursively quicksort the left partition.
5. Recursively quicksort the right partition.
6. Combine implicitly (partitions are sorted in place).

**How it works :**

1. Choose a pivot element (commonly last element, first element, median, or randomized).

2. Partition the array so that elements less than pivot are on the left and greater on the right; the pivot ends up in its final sorted position.

3. Recursively quicksort left partition and right partition.

**Partitioning idea :** maintain a boundary index of smaller elements; as you scan the array, swap elements to the left partition when they are ≤ pivot; after scan, place pivot after boundary.

**Worked example:** `[10, 80, 30, 90, 40, 50, 70]` (choose pivot = `70`)

- Partition step moves `10,30,40,50` before pivot and `80,90` after pivot → partitions `[10,30,40,50] [70] [80,90]`

- Recursively sort left and right partitions → final sorted array `[10,30,40,50,70,80,90]`.

**Complexity & notes:**

- Average time: **Θ(n log n)**.

- Worst case: **O(n²)** (when partitions are highly unbalanced, e.g., sorted input and pivot always maximum/minimum).

- Space: average **O(log n)** recursion depth, worst **O(n)**.

Syntax Syndicate

- Practical tips: randomize pivot or use median-of-three to avoid worst case; typically very fast in practice.

---

## Binary Search

### Binary Search — Algorithm

1. Ensure the array is sorted.
2. Set low = 0, high = n − 1.
3. While low ≤ high:
    - Compute mid = floor((low + high)/2).
    - If array[mid] == target → found, stop.
    - If target < array[mid] → set high = mid − 1 (search left).
    - Else set low = mid + 1 (search right).
4. If loop ends without finding target, target not present.

**How it works :**
Binary search requires a sorted array. Compare the target key with the middle element. If equal → found. If target < middle → continue search in left half. If target > middle → continue in right half. Repeat until found or subarray empty.

**Worked example:** On `[10, 20, 30, 40, 50]`, search for `30`: middle element is `30` — immediate success.

**Complexity & notes:**

- Time: **Θ(log n)**.
- Space: **O(1)** iterative. Always ensure input is sorted.

---

## Finding Minimum & Maximum with Divide & Conquer

**Naive approach :** scan array once, maintain `min` and `max` — requires `2(n-1)` comparisons.

**Divide & Conquer approach :**

- Split array into two halves; recursively find `min` & `max` for each half; then compare the two minima and the two maxima to get global `min` and `max`.

Syntax Syndicate

- This reduces comparisons to approximately `3 ⌈n/2⌉ − 2` (about `1.5 n`) which is better than `2n` for large `n`.

**Worked small example:** `[7, 2, 9, 4]`

- Left half `[7,2]` → min=2, max=7 (1 comparison)
- Right half `[9,4]` → min=4, max=9 (1 comparison)
- Compare minima `2 vs 4` (1 comparison) and maxima `7 vs 9` (1 comparison) → total 4 comparisons vs naive 6.

**Practical note:** Use divide & conquer when comparisons cost is significant; otherwise linear scan is simpler.

---

# Unit III — Greedy Method & Graph Algorithms

## Greedy Paradigm (overview)

**Intuition / When to use:**
A greedy algorithm makes the locally best choice at each step, hoping to reach a global optimum. Use when problem has the **greedy-choice property** (local optimum leads to global optimum) and **optimal substructure**.

**Typical approach :**

1. Define a measure of “best” at each step (e.g., smallest weight, highest value/weight ratio).
2. Make the greedy choice and reduce the problem (remove chosen item or mark covered).
3. Continue until solution is built.
4. Prove correctness by exchange argument or induction (show greedy choice can be extended to an optimal solution).

**Important:** Greedy works for some problems (MST, fractional knapsack) but fails for others (0/1 knapsack usually needs DP).

---

Syntax Syndicate

# Dijkstra's Algorithm

## Dijkstra's Algorithm — Algorithm

1. Initialize distance to source = 0; distances to all other vertices = ∞. Mark all vertices as unvisited.
2. While there are unvisited vertices:
    - Select the unvisited vertex u with smallest tentative distance.
    - Mark u as visited (finalize distance).
    - For each neighbor v of u that is unvisited:
        - Compute alt = distance[u] + weight(u,v).
        - If alt < distance[v], set distance[v] = alt (update).
3. Repeat until all reachable vertices finalized.
4. The distance table now holds shortest distances from source.

**What it solves:** Shortest paths from a single source to all vertices in a graph with **nonnegative edge weights**.

**Table method :**

**Example Graph:**
Vertices = {A, B, C, D, E}
Edges with weights:
A–B=4, A–C=2, B–C=5, B–D=10, C–E=3, D–E=4

**Table Representation**

| Step | Chosen Vertex | Distance from A | Updated Distances | Final Distances |
|---|---|---|---|---|
| Init | - | A=0, others=∞ | - | A=0, B=∞, C=∞, D=∞, E=∞ |
| 1 | A | 0 | B=4, C=2 | A=0, B=4, C=2, D=∞, E=∞ |

Syntax Syndicate

| | | | | |
|---|---|---|---|---|
| 2 | C | 2 | E=2+3=5 | A=0, B=4, C=2, D=∞, E=5 |
| 3 | B | 4 | D=4+10=14 (not updated since later better path) | A=0, B=4, C=2, D=14, E=5 |
| 4 | E | 5 | D=min(14, 5+4=9)=9 | A=0, B=4, C=2, D=9, E=5 |
| 5 | D | 9 | - | Final: A=0, B=4, C=2, D=9, E=5 |

**How it works :**

1. Initialize distances: `distance[source] = 0`, others = ∞.
2. Maintain a set of visited (finalized) vertices whose shortest distance is known.
3. Repeatedly pick the unvisited vertex with the smallest tentative distance. Mark it visited.
4. For each neighbor of that vertex, relax the edge: if going through the new vertex gives a smaller distance, update the neighbor's tentative distance.
5. Stop when all vertices are visited or no reachable unvisited vertices remain.

**Complexity & notes:**

- With adjacency matrix and linear search for min vertex: **$O(V^2)$**.
- With min-heap (binary heap) and adjacency lists: **O(E log V)**.
- Cannot handle negative edge weights — use Bellman-Ford for those.

---

Syntax Syndicate

# Fractional Knapsack

## Fractional Knapsack — Algorithm

1. For each item compute value/weight ratio (value per unit weight).
2. Sort items in descending order of ratio.
3. Initialize remaining capacity = W, total value = 0.
4. For each item in sorted order:
    - If item weight ≤ remaining capacity: take whole item, subtract weight, add full value.
    - Else take fraction = remaining capacity / item weight, add fractional value and set remaining capacity = 0; stop.
5. Output total value and selected fractions.
6.

**What it solves:** Maximize total value with capacity constraint when items can be split (take fractions).

**How it works :**

1. Compute value/weight ratio for each item.

2. Sort items by descending ratio (highest value per unit weight).

3. Take items in that order: if whole item fits, take it; if it doesn't, take the fractional part that fits and stop.

- **Example:**
- Items = { (Profit=60, Weight=10), (Profit=100, Weight=20), (Profit=120, Weight=30) },
- Capacity = 50.

| Item | Profit | Weight | Profit/Weight | Take | Remaining Capacity |
|---|---|---|---|---|---|
| 1 | 60 | 10 | 6 | Full (10) | 40 |
| 2 | 100 | 20 | 5 | Full (20) | 20 |
| 3 | 120 | 30 | 4 | 20/30 = 2/3 | 0 |

Syntax Syndicate

Total Profit = 60 + 100 + (120 × 20/30) = 240.

**Complexity & notes:**

- Sorting dominates → **O(n log n)**.
- Greedy is optimal for fractional knapsack (exchange argument).

---

## Minimum Spanning Trees — Kruskal & Prim

**Problem:** Given an undirected weighted connected graph, find a subset of edges connecting all vertices with minimum total weight and no cycles.

**Kruskal's approach :**

### Kruskal's Algorithm

1. Create a list of all edges and sort them by increasing weight.
2. Initialize MST as empty and create Union-Find structure with each vertex in its own set.
3. For each edge (u,v) in sorted list:
    a. If u and v belong to different sets (find(u) ≠ find(v)):
        i. Add edge (u,v) to MST.
        ii. Union the sets containing u and v.
    b. Else skip (would form a cycle).
4. Stop when MST has V − 1 edges.

**Notes: sorting O(E log E); union-find operations nearly linear. Good for sparse graphs.**

**Edges Sorted by Weight:**

( A–C=1, B–C=2, C–D=3, B–D=4, A–B=5 )

Syntax Syndicate

| Step | Edge Considered | Added to MST? | MST Edges | Total Weight |
|---|---|---|---|---|
| 1 | A–C (1) | Yes | {A–C} | 1 |
| 2 | B–C (2) | Yes | {A–C, B–C} | 3 |
| 3 | C–D (3) | Yes | {A–C, B–C, C–D} | 6 |
| 4 | B–D (4) | No (Cycle) | Same | 6 |
| 5 | A–B (5) | No (Cycle) | Same | 6 |

Final MST Weight = **6**

**Prim's approach :**

## Prim's Algorithm

1. Choose a start vertex; add it to MST set.
2. Initialize a list/priority structure of candidate edges from the MST set to non-MST vertices (or maintain best edge to each non-MST vertex).
3. Repeatedly:
    - Select the minimum-weight edge that connects MST set to a vertex not yet in MST.
    - Add that edge and the new vertex to MST.
    - Update candidate edges (or best edges to each outside vertex) based on the newly added vertex.
4. Repeat until MST contains all vertices.

**Edges Sorted by Weight:**

( A–C=1, B–C=2, C–D=3, B–D=4, A–B=5 )

Starting Vertex = A

Syntax Syndicate

| Step | Chosen Edge | MST Set | Total Weight |
|---|---|---|---|
| Init | - | {A} | 0 |
| 1 | A–C (1) | {A,C} | 1 |
| 2 | C–B (2) | {A,C,B} | 3 |
| 3 | C–D (3) | {A,C,B,D} | 6 |

Final MST Weight = **6**

**Complexity & notes:**

- Kruskal: sorting edges → **O(n log n)**; union-find nearly linear in n.
- Prim: with adjacency list + binary heap → **O(n log n)**.
- Kruskal is good for sparse graphs; Prim often better for dense graphs. Both produce MSTs and are greedy but use different greedy choices (edge-based vs vertex-based).

---

## Final exam tips (how to write answers that get full marks)

1. Always start with a one-line definition/intention (what problem does the algorithm solve).
2. Describe the **approach in words** (divide & conquer / greedy / DP) and why that is appropriate.
3. Give a **small worked example** (2–8 elements) and show the intermediate states — examiners like to see you understand the steps.

Syntax Syndicate

4. State **time & space complexity** with reasoning. If possible, give best/worst/average.

5. Mention **practical notes**: stability, when to use, pitfalls (e.g., quicksort worst case, Dijkstra negative edges).

6. If asked to prove correctness, use an **exchange argument** for greedy or **induction** for DP/recurrence solutions.

Syntax Syndicate